

Advanced Software Development

Architectural Design



Simon H. Muchinenyika

Architectural Design

Table of Contents

- Introduction
 - Separation of Concerns, KISS
- Architectural Design
 - MVA, Minimum Vs Viable
- Group Assignment
 - Deliverables, Groups, Timelines
- Conclusion

Architectural Design

Introduction

- code written a few months ago!
- software has to be soft despite the size of the project;
- software product Vs software project;
- will the product scale up when need arise?
- software architecture communicates the intention of the system.



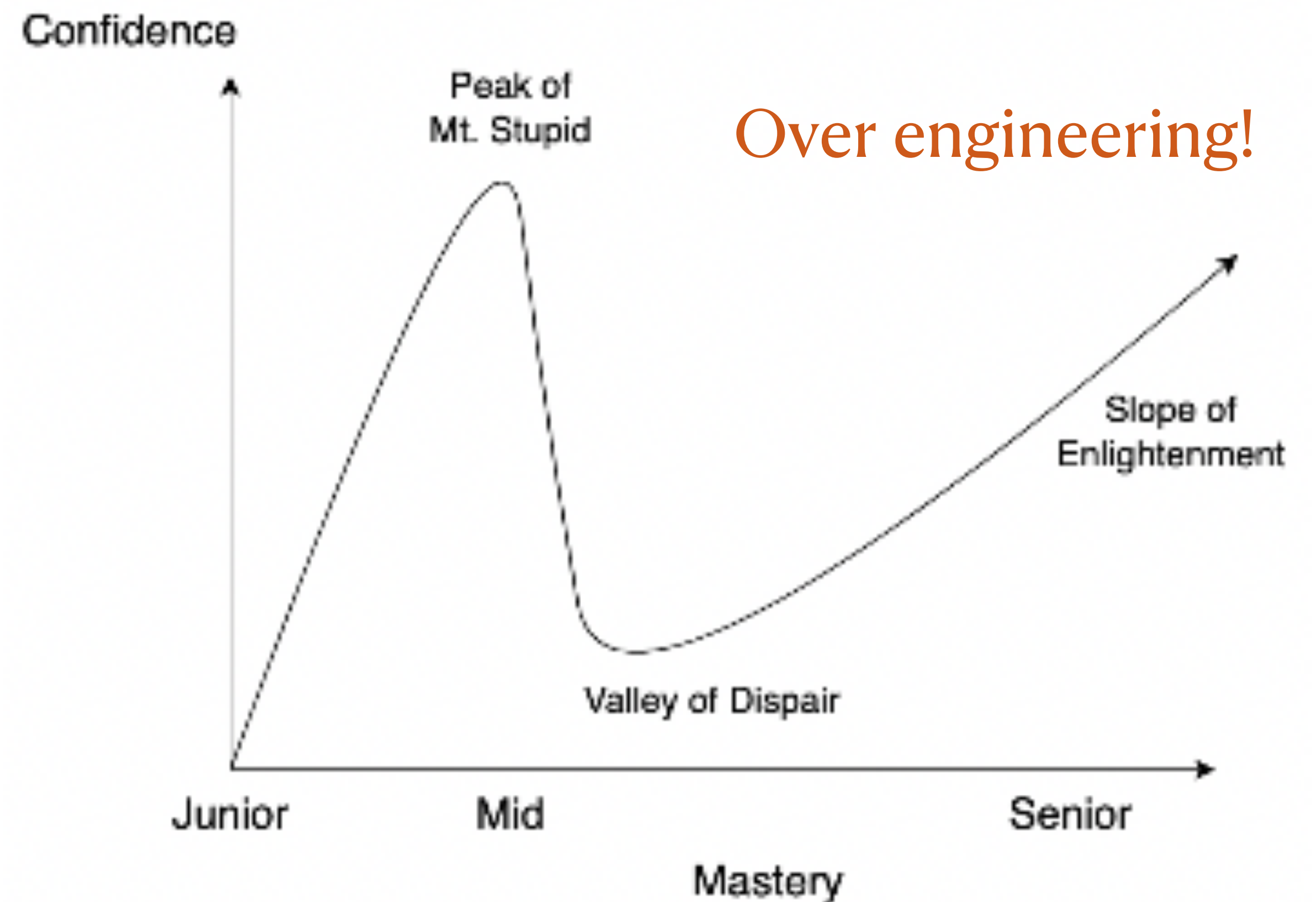
Architectural Design

Introduction

- Coupling:
 - clear separation of concern;
 - logical cohesion.
- Class size: related responsibilities?
- Automated test:
 - both unit and instrumentation tests.
- Defined architectural pattern.

WHAT TO AVOID

Over engineering!



Architectural Design

Separation of Concerns

- a concern is a set of information that affects the program's code.
- program that embodies SoC is a layered design, modular, or a form of abstraction.
- modularity is attained by encapsulation & common layered designs have presentation, business logic, data access, & persistence layers.
- SoC brings more freedom of the program's design, deployment, & usage.
- Examples:
 - Internet Protocol Stack;
 - HTML, CSS, & JS.

Architectural Design

KISS

- code becomes easy to understand,
- easy to refactor without breaking functionality, and
- easy add new functionality.

```
fun mapColorForOwedAmount(amount: Int): Int {  
    if (amount in 0..49)  
        return R.color.green  
    else if (amount in 50..99)  
        return R.color.yellow  
    else if (amount in 100..199)  
        return R.color.orange  
    else  
        return R.color.red  
}
```

```
fun mapColorForOwedAmount(amount: Int): Int {  
    return when (amount) {  
        in 0..49 -> R.color.green  
        in 50..99 -> R.color.yellow  
        in 100..199 -> R.color.orange  
        else -> R.color.red  
    }  
}
```

Architectural Design

First things first

- upon establishing the requirements, selecting a development stack, & agreeing on a release cycle, other factors to consider:
- the development team:
 - size, level of expertise, number of teams, retention.
- amount and complexity of features:
 - what type of logic, how complex, how many features, how do they interact, dependencies, APIs exposed, backend requirements?
- product longevity? (Important to distinguish product from project)

Architectural Design

Minimum Viable Architecture (MVA)

- Minimum Viable Product (MVP) - the simplest product that makes sense to be released to validate an idea;
- derived from Lean Startup where learning is done in the development of the new product.
- MVA is the simplest architecture that can support clean product implementation.
- **Minimum:** avoid speculation of future requirements, e.g., segregating the code base into feature modules.
- **Viable:** architecture should support development of the current scope while respecting SOLID principles, clean architecture, and other good practices.

Architectural Design

Minimum Vs Viable

- clearly there is a tradeoff between minimum and viable in MVA.
- Which one leads to over-engineering and which one to under-engineering?
- How best do you think this tradeoff should be addressed?
- the key to mastery is the ability to balance the two.
- balance evolves over time as the MVA gets complex. What was viable at one point ceases to be viable
- readjustments may be necessary at every iteration of design or analysis.

Architectural Design

Example Revisited

```
fun mapColorForOwedAmount(amount: Int, customer: Customer): Int {  
    if (amount in 0..49) {  
        if (customer.type == CustomerType.PREMIUM ||  
            customer.type == CustomerType.VIP)  
            return R.color.green  
        else  
            return R.color.yellow  
    } else if (amount in 50..99)  
        if (customer.type != CustomerType.VIP) {  
            if (customer.balance > 200)  
                return R.color.yellow  
            else  
                if (customer.type == CustomerType.GUEST)  
                    return R.color.red  
                else  
                    return R.color.orange  
        } else  
            return R.color.yellow  
}
```

```
    else if (amount in 100..199)  
        if (customer.balance < 100) {  
            if (customer.type == CustomerType.PREMIUM ||  
                customer.type == CustomerType.VIP)  
                return R.color.orange  
            else  
                return R.color.red  
        } else  
            return R.color.orange  
    else {  
        if (customer.type == CustomerType.VIP &&  
            customer.balance > 100)  
            return R.color.orange  
        return R.color.red  
    }  
}
```

Two more rules added, but violated the Open-Closed Principle.

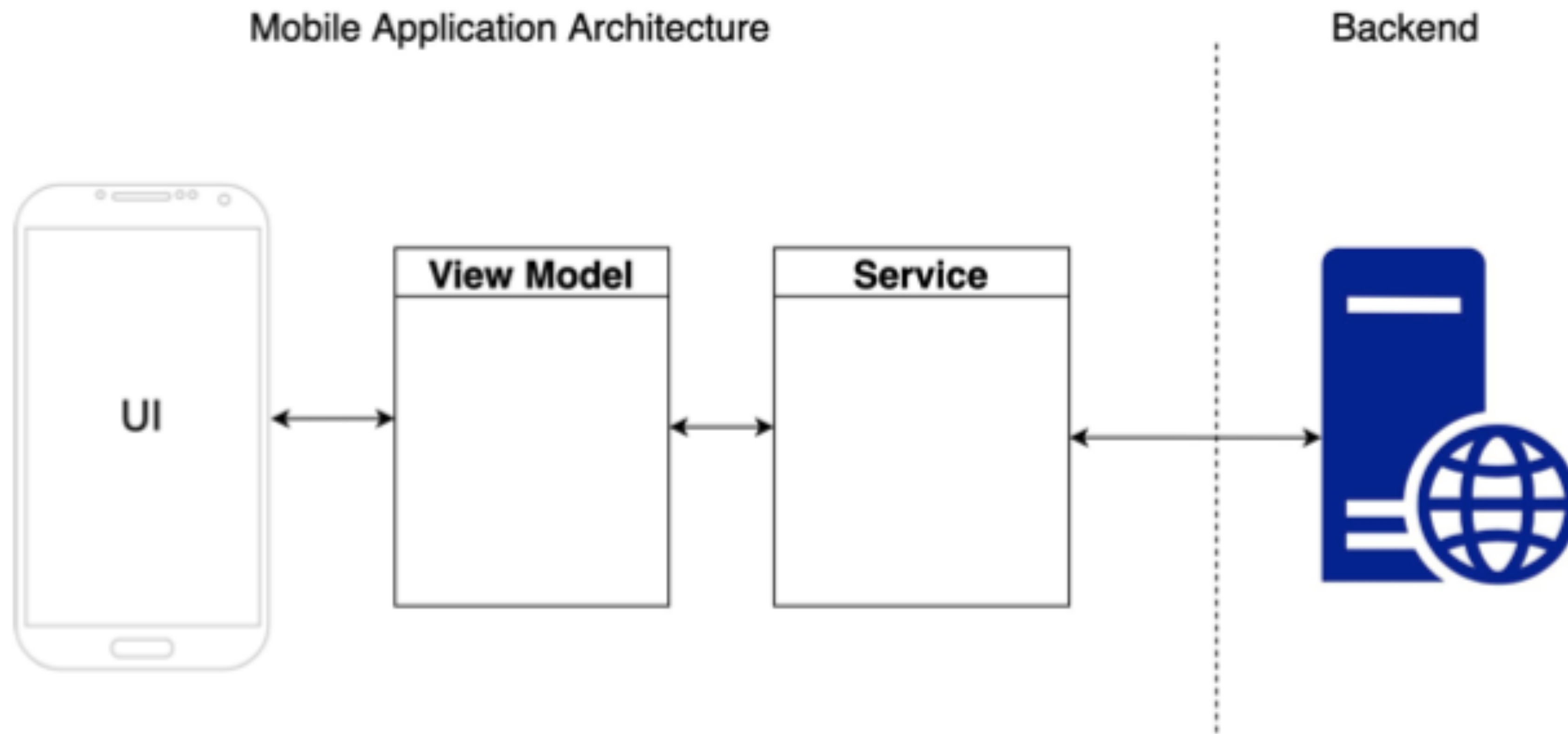
Architectural Design

Right Time to Refactor

- no one size fit all:
 - overengineering is based on speculation, while the other extreme under-engineers;
- So when is the right time to refactor?
 1. while reviewing our own implementation before commitment.
 2. when a colleague complains that the code is hard to understand during team reviews.
 3. after receiving a new requirement. It's more efficient to refactor before adding new functionality.

Architectural Design

The Simplest MVA Example for CRUD Applications



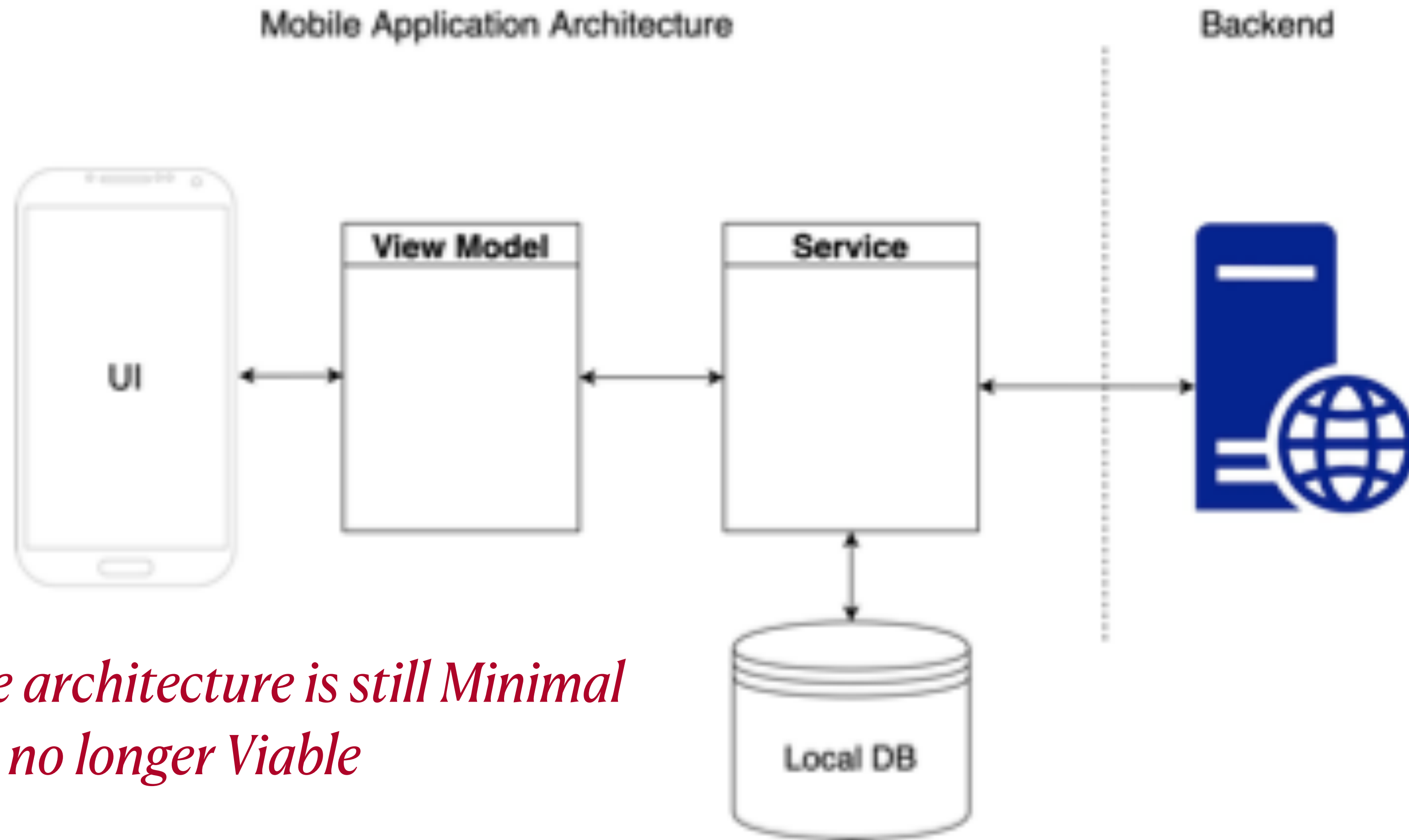
Architectural Design

MVA Example Continued

- let's suppose we get feedback that is not favourable after getting the MVP out:
 - users want the application to be offline;
 - delays experienced due to backend performance issues;
- suppose HTTP caching is not sufficient to address performance issues, how else could you address this challenge?

Architectural Design

MVA With A Bloated Service

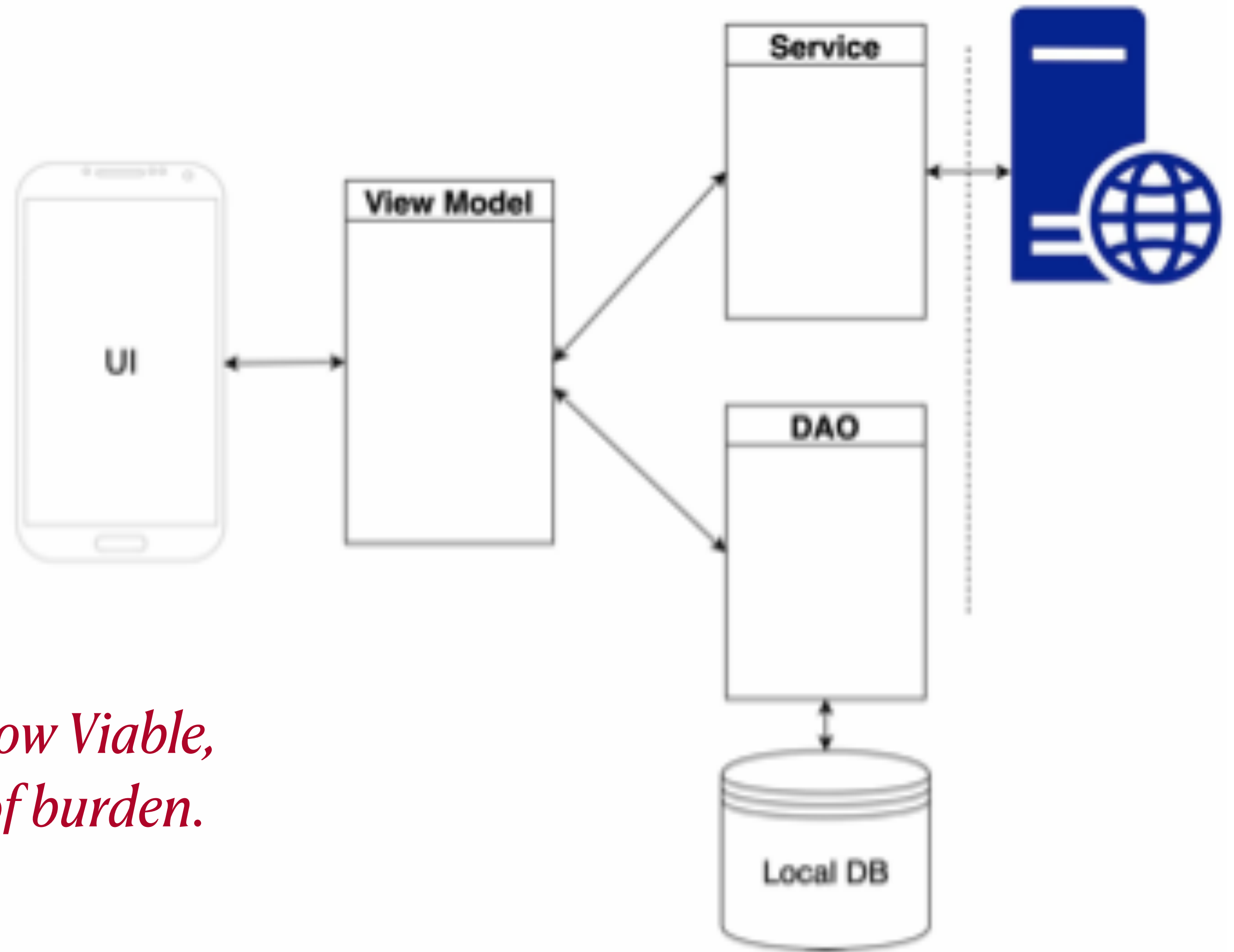


*Even though the architecture is still Minimal
It is no longer Viable*

Architectural Design

MVA With A Bloated View Model

- to make the architecture Viable, we can introduce another layer to handle the database operations.
- the DAO layer is now responsible for all interactions with our local database.

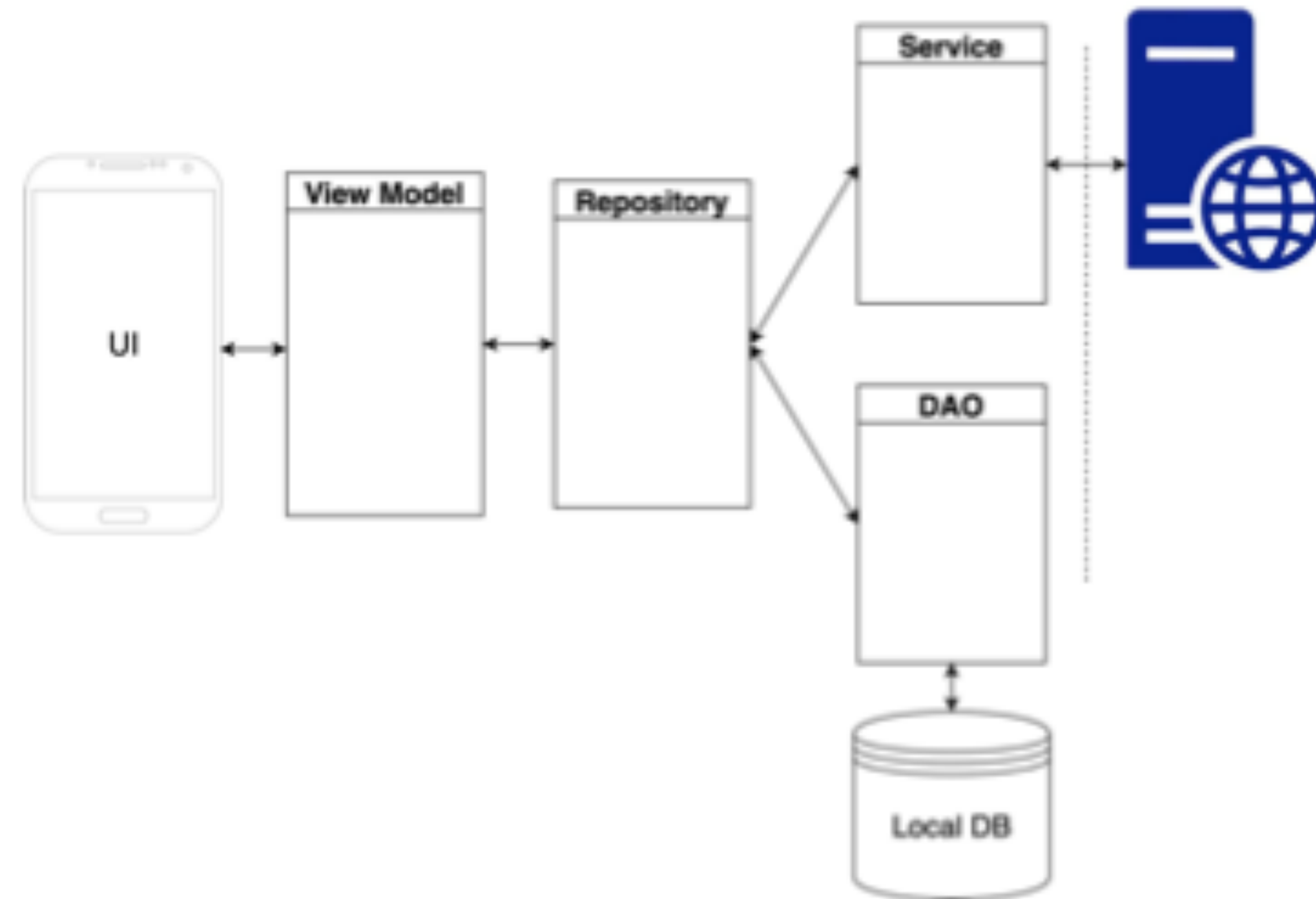


*Even though the architecture is now Viable,
The View Model now takes a lot of burden.*

Architectural Design

MVA With Offline Support

- introducing another layer takes the concern away and makes MVA viable again;
- the Repository has only one concern of deciding whether data should come from the DAO or/and the service.
- all caching and offline rules can be implemented.



Architectural Design

Increasing Complexity

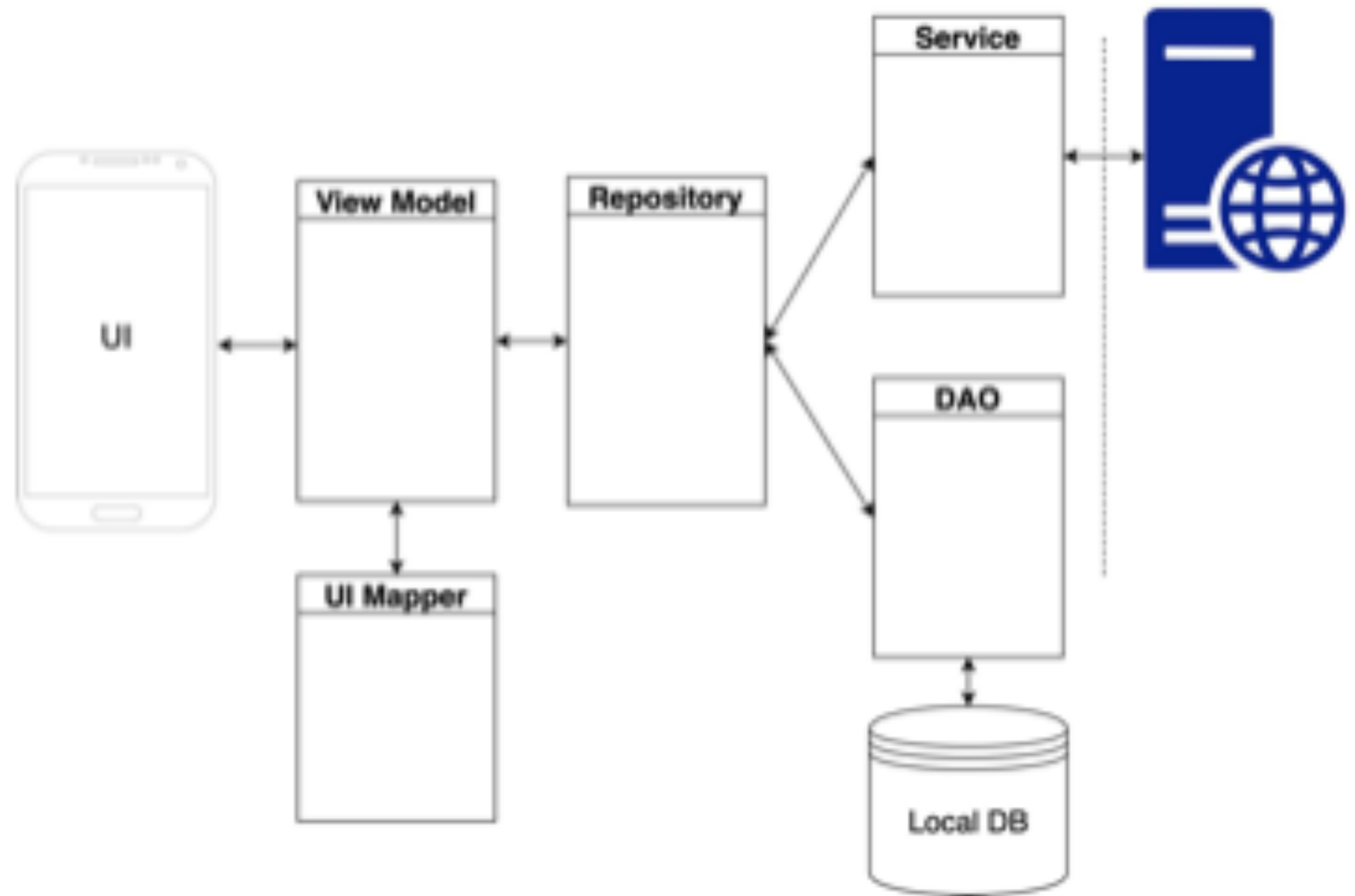
- suppose our users are now happy with the caching & offline capabilities.
- however, the Product Owner has new requirements, including making UI decisions based on data;
- Example:
Changing the screen element's color based on the owed amount. **How would this affect your architecture?**
- Note that adding the logic inside the View Model would soon blot the layer.

Architectural Design

Another Layer ...

- introduce transformation mappers to apply the presentation logic.
- Consider a Customer model as an example:

```
struct Customer {  
  var id: String  
  var fullName: String  
  var phoneNumber: String  
  var address: String  
  var balance: Int  
  var customerType: CustomerType  
  var owedAmount: Int  
}
```



Architectural Design

Mapped Customer Model

```
struct CustomerPresentation {  
    var id: String  
    var fullName: String  
    var balance: Int  
    var balanceColor: String  
    var balanceFontSize: Int  
    var customerType: CustomerType  
    var owedAmount: Int  
    var owedAmountColor: String  
    var owedAmountFontSize: Int  
}
```

- take note the new model has required customer info and additional fields:
 - owedAmountColor;
 - owedAmountFontSize.
- fields not displayed are also removed.
- such an approach does not violate the Interface Segregation Principle.
- code becomes much less like:

if (customer.owedAmount > 400)

customerOwedAmountColor = colors.RED

Architectural Design

Summary

- the layers discussed means every *Entity* should have its own Service, DAO, and Repository;
- each screen should have its UI and View Model.
- the challenge with this 1-1 relationship of the UI and data, is that often times the screen requires combined information
- this means building a data layer around a data topology and the presentation layer around UI/UX makes MVA not viable when data aggregation is required.
- As homework, refactor the MVA presented on slide 18 to make it aggregate data.

Architectural Design

Further Study

- S.O.L.I.D Principles

Architectural Design

References

- Ahmed, S., Bloom, I., Jansen, A. N., Tanim, T., Dufek, E. J., Pesaran, A., Burnham, A., Carlson, R. B., Dias, F., Hardy, K., Keyser, M., Kreuzer, C., Markel, A., Meintz, A., Michelbacher, C., Mohanpurkar, M., Nelson, P. A., Robertson, D. C., Scofield, D., ... Zhang, J. (2017). Enabling fast charging – a battery technology gap assessment. *Journal of Power Sources*, 367, 250–262. <https://doi.org/https://doi.org/10.1016/j.jpowsour.2017.06.055>
- Android Developer: <https://developer.android.com/topic/architecture>
- Efthymious, P. (2022). Clean Mobile Architecture: Become an Android, iOS, Flutter Architect. John Wiley & Sons, ISBN-10: 6188600731
- Separation of Concerns: https://en.wikipedia.org/wiki/Separation_of_concerns

Architectural Design

Thank You

Questions?

T: +264 61 207 2054

F: +264 61 207 9054

E: smuchinenyika@nust.na